

COMPSYS 723 Assignment 2 Report

Liam Hall and Inesh Bhanuka

*Department of Electrical, Computer Systems and Software Engineering
University of Auckland, Auckland, New Zealand*

Abstract—This report presents the design and implementation of a deterministic, simplified cruise control system for a semi-autonomous vehicle using the synchronous reactive programming language Esterel V6. Operating within a safe velocity bound of 30km/h to 150km/h, the system dynamically manages four discrete execution states: OFF, ON, STDBY, and DISABLE, while regulating vehicle speed via a proportional-integral (PI) control loop ($K_p = 8.113$, $K_i = 0.5$). Adopting a model-based design flow, the controller architecture is decomposed into five concurrent submodules running in parallel under a single top-level parent entity: PedalDetection, CruiseSpeedCheck, CruiseStateHandler, cruise_speed_management, and car_driving_control.

To optimize performance and safety, a hybrid architecture was deployed, combining Esterel’s precise temporal preemption primitives (abort, sustain) for state transitions with embedded C function-call integration to manage complex mathematical data handling and error-term resets within the PI control loop. Critical safety-oriented engineering decisions were incorporated into the design, including treating absent pedal inputs as zero to mitigate hardware disconnection faults and discarding all input data on the initial clock tick to insulate the state machine from startup electrical transients. Furthermore, strict signal dependency mapping was used to ensure complete architectural causality and to pass compile-time race condition checks without introducing artificial processing delays. System correctness was validated through independent modular evaluation followed by top-level regression testing using automated simulator input and output vectors (vectors.in/vectors.out), verifying total conformance to the operational specification.

Index Terms—Scientific writing, Typesetting, Document creation, Syntax

I. INTRODUCTION

This report presents the design and implementation of a simplified cruise control system. The system is implemented using Esterel V6, a synchronous reactive programming language well-suited for embedded control systems that must respond deterministically to concurrent inputs.

The cruise control system is designed to maintain a vehicle at a constant speed without driver input, operating within a defined speed range of 30.0 km/h to 150.0 km/h. The system manages four discrete control states, OFF, ON, STANDBY, and DISABLE, and regulates vehicle speed using a proportional-integral (PI) control algorithm with gain parameters $K_p = 8.113$ and $K_i = 0.5$.

The design process followed a model-based approach, beginning with the development of a system context diagram to identify system boundaries and interfaces. A data-flow and control-flow decomposition was then performed to separate the system into concurrent functional modules: pedal detection, speed validation, cruise state management, cruise speed management, and throttle regulation. The behavior of

each module was captured using finite-state machines prior to implementation, ensuring a traceable mapping from the specification to the code.

The final implementation consists of a set of Esterel modules, composed in parallel under a top-level controller, reflecting the system’s concurrent and hierarchical requirements. Correctness of the implementation was verified against the provided input and output test vectors.

II. FORMAL SPECIFICATION DIAGRAMS

The first step taken towards implementing the cruise controller system in Esterel was designing a context diagram to capture the desired functionality. Creating design diagrams enables complete visualization of inputs and outputs, along with the system’s control flow, ensuring a smooth development process.

A. Context Diagram

Designed the context diagram, which helped visualize the inputs and outputs of the cruise controller, as shown in Fig. 1. The system receives both sensor data inputs and control signals, represented in Esterel as valued float signals and pure signals, respectively. The system provides the system state as an enumeration, as well as two-valued float signals for the cruise speed and throttle command.

Inputs:

- On
 - Pure Signal: Enables cruise control
- QuickAccel
 - Pure Signal: Increases cruise speed
- Off
 - Pure Signal: Disables cruise control
- QuickDecel
 - Pure Signal: Decreases cruise speed
- Resume
 - Pure Signal: Re-enables cruise control after braking
- Accel
 - Valued Float Signal: Accelerator pedal sensor
- Set
 - Pure Signal: Sets the current speed as the cruise speed
- Brake
 - Valued Float Signal: Brake pedal sensor
- Speed
 - Valued Float Signal: Speed sensor of the vehicle

Outputs:

- CruiseState

- Enumeration (Implemented as Integer): The state of the cruise controller (OFF (1), ON (2), STDBY (3), DISABLE (4))
- CruiseSpeed
 - Valued Float Signal: Cruise speed value
- ThrottleCmd - Valued Float Signal: Throttle command

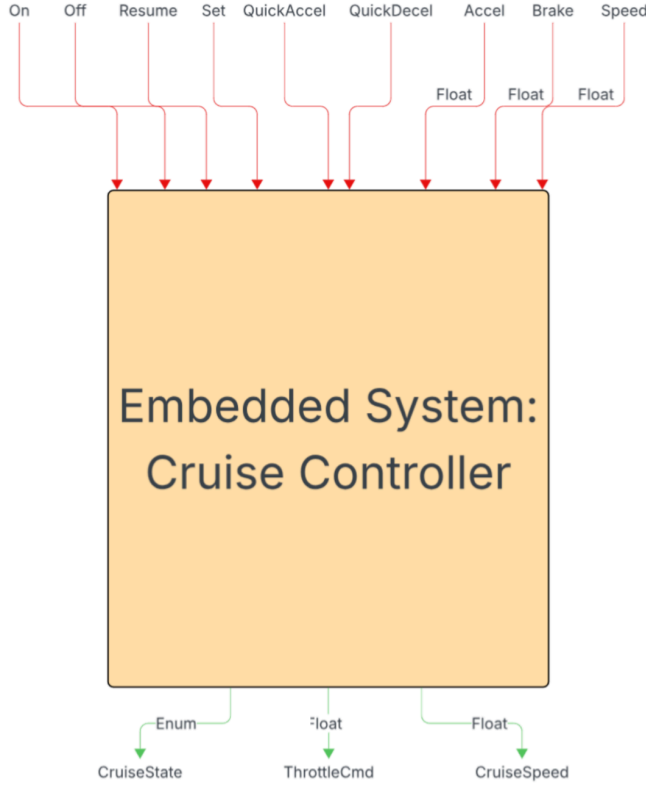


Fig. 1. Cruise Controller Context Diagram

B. First Level Refinement Diagram

The first level refinement diagram shown in Fig. 2 shows how the cruise controller design was divided into five submodules: PedalDetection, CruiseSpeedCheck, CruiseStateHandler, cruise_speed_management, and car_driving_control. These submodules are all executed in parallel, within one parent module. The key benefit of designing a first level refinement diagram is being able to visualize how the input signals to the system from the environment, along with the internally generated signals are connected internally to produce the desired output signals.

The internal only signals of the system are: Accel_Active, Brake_Active and InvalidCruiseSpeed. These signals are generated within the PedalDetection and CruiseSpeedCheck modules that help with the functionality of the other main modules that produce the desired outputs.

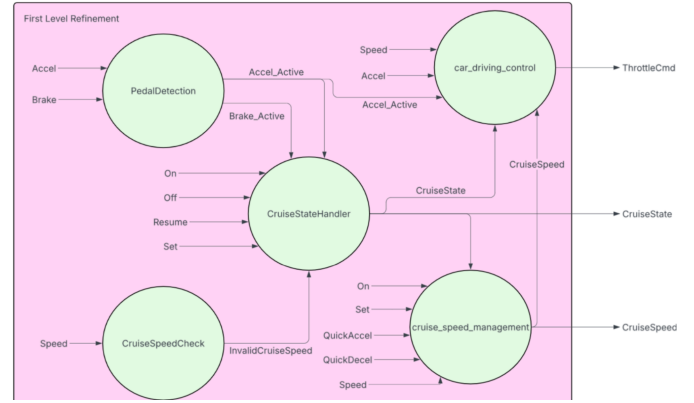


Fig. 2. Cruise Controller First-Level Refinement Diagram

III. FINITE STATE MACHINES

A. Pedal Detection

The Pedal Detection module is a helper module, described as an FSM in Fig. 3, detects when the accelerator or brake pedals have been pressed. To account for unintentional disturbances that may occur on the accelerator or brake pedals, the pedal sensors must reach a certain threshold value PEDALSMIN before an acceleration or brake input is registered. This module compares whether the Accel or Brake sensor input values have exceeded the minimum threshold for a valid press on the pedal and emits the corresponding Accel_Active or Brake_Active internal signals.

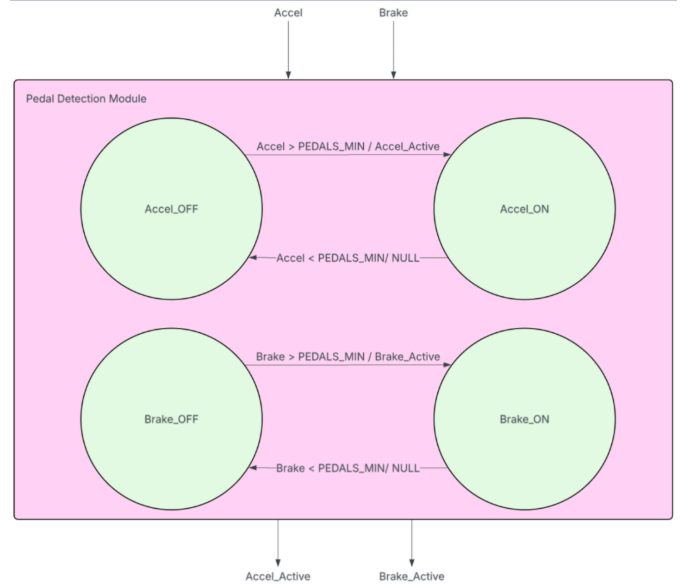


Fig. 3. Pedal Detection FSM

B. Cruise Speed Check

The CruiseSpeedCheck module is another helper module that determines whether the current speed of the car is outside the allowed limits for cruise speed. The module compares the input value Speed with the threshold values SPEEDMIN and SPEEDMAX, if the speed is outside the valid bounds, the

module will emit the InvalidCruiseSpeed internal signal. The FSM for this module is described in Fig. 4.

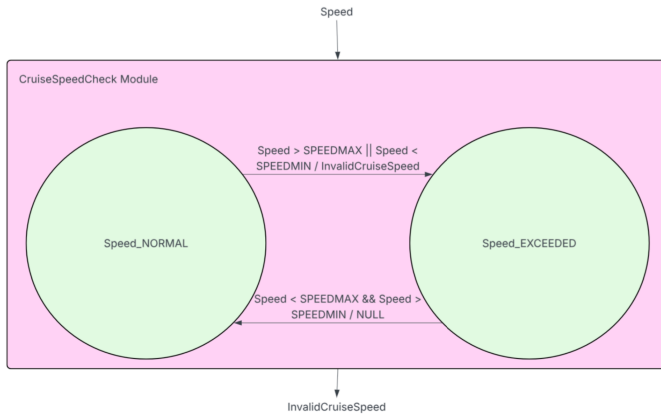


Fig. 4. CruiseSpeedCheck FSM

C. Cruise State Handler

The CruiseStateHandler module defines what operating state the cruise control system is in. It implements a deterministic finite state machine (FSM) that manages the operational mode of the vehicle based on driver commands (On, Off, Resume) and safety flags (Accel_Active, Brake_Active and InvalidCruiseSpeed) generated by the helper modules. The FSM for the CruiseStateHandler is described in Fig. 5.

The module enumerates four distinct operational modes, mapped to an integer output signal CruiseState:

- OFF_State: System is completely inactive.
- ON_State: System actively regulates vehicle speed.
- STDBY_State: System is temporarily paused due to brake engagement.
- DISABLE_State: System control is overridden due to safe limit violations or accelerator use.

The system is initialized to OFF on startup, and only activates when the On signal is present. If there is a brake input while the system is in the ON state, determined by the Pedal Detection module, the system immediately transitions to a STDBY state, where it does not regulate speed and waits for a Resume signal. During the ON state, if there is acceleration input (determined by the Pedal Detection module as well) or the current speed has violated the safety thresholds, determined by the CruiseSpeedCheck module, then the system is set to the DISABLE state. The system will only return to the On state when both the accelerator is not pressed and the current speed is within the safety thresholds.

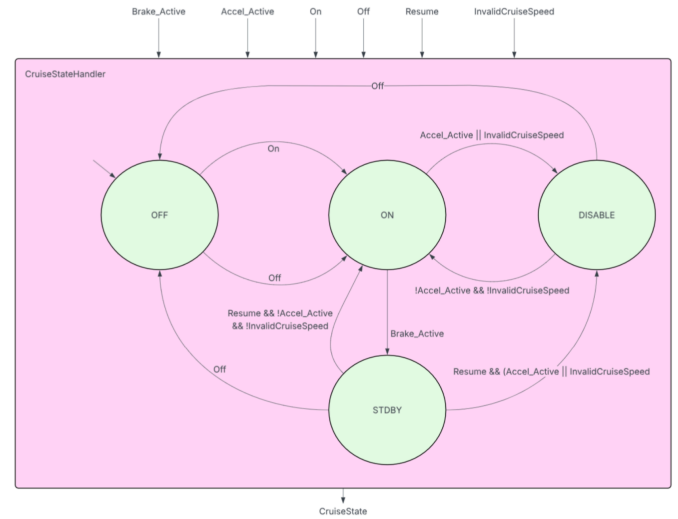


Fig. 5. CruiseStateHandler FSM

D. Car Driving Control

The car_driving_control module is in charge of generating the ThrottleCmd output for the car, this output then used by the car to determine how much to accelerate or decelerate at each time step. When the cruise controller first becomes active, signified by CruiseState being “On”, this module calls a function in C which sends relevant information to the PI controller [1], clearing the accumulating error term, and returning the controller’s response. Any subsequent ticks that the cruise controller is active, the C function will be called, accumulating the error term, and automatically controlling the car’s speed. As soon as the CruiseState is no longer “On”, this module will stop automatically regulating the car’s speed, clearing the controller error term, and the ThrottleCmd will be simply driven by the car’s accelerator pedal. The FSM is described in Fig. 6.

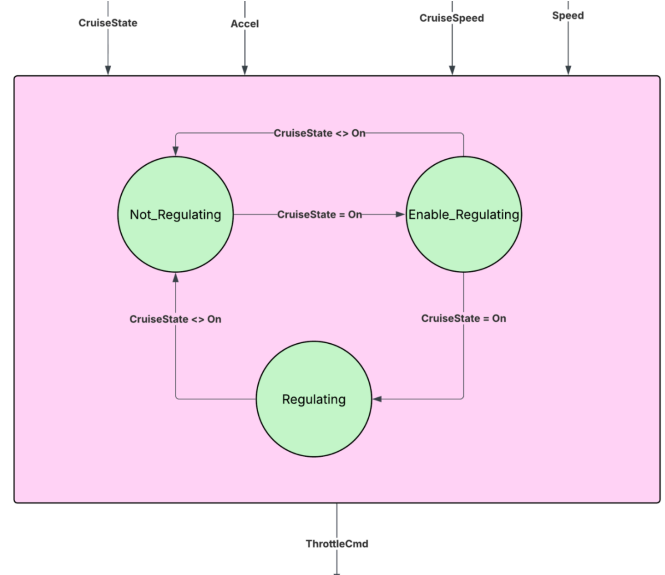


Fig. 6. car_driving_control FSM

E. Cruise Speed Management

For the `Cruise_speed_management` module a finite state machine is not necessary to describe its behaviour, as it takes only one state. In each tick, it checks whether any of the input conditions have been met; each tick, as long as `CruiseState` is not “Off” it will increment `CruiseSpeed` by one configurable unit when `QuickAccel` is active, decrement `Cruise speed` when `QuickDecel` is active and set the `CruiseSpeed` to `Speed` when `Set` is active. It stores the current value of `CruiseSpeed` between ticks in order to eliminate the need for it to read this value each time it needs.

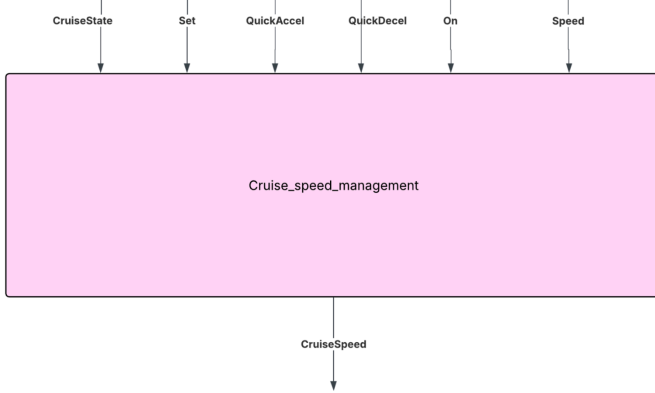


Fig. 7. `cruise_speed_management` FSM

IV. CAUSALITY HANDLING

Throughout this project, we had many signals shared between modules, and such, we had to ensure that the way we use these signals is causal. To do this, we carefully constructed our program such that no non-deterministic behavior forms, and there exist no cyclic loops to void the program’s determinability.

One way we prevented any such behavior from occurring is by ensuring no two processes both depended off the behavior of the other. For example, many of our nodes must read the current values of `CruiseState`, due to this, the `CruiseState-Handler` must not depend on any of the nodes that depend on it. This method allows the program to run without excessive use of the esterel function “`pre()`”. Although this function is essential for certain types of behaviour to be possible, it introduces a tick of unnecessary delay.

To test the causality of the system we used the esterel compile-time causality assessment, a process in which the system runs a rigid set of tests to determine whether the program can execute with any race conditions or non-deterministic behaviour. Although this test must pass to compile, we specifically used it in assessment of each of our modules to accurately and easily detect potential issues in our logic.

V. TESTING

To test our design we used the esterel simulator to simulate the external stimuli from the car to ensure our functionality is up to standard. Initially we tested each of our components independently so we could have a simpler view of debugging

each component to isolate any issues it may have. This process was first done by inspection, manually inputting signals into the simulator, and checking the right result was produced. After this manual process, we used a set of automated tests by replacing the stdin input stream with a `vectors.in` file, and the stdout stream with a `vectors.out` file. This allows us to follow a more thorough testing process and find any edge cases that may exist before declaring a component as functional.

After each component was tested individually, we connected each component to a top-level entity, which simply ran them in parallel, while routing any internal signals between each component. The same manual followed by the automatic testing process was repeated at this stage with a higher level of scrutiny to ensure the design was fully up to specification.

VI. DESIGN DECISIONS

One major decision we made throughout this project was how we segmented the functionality of our system. We decided to split the functionality into five major parts as shown in the above diagrams, this modular style design was chosen due to its simplicity in terms of debugging and visual understanding. With this method, we could easily test each component individually, to ensure that each part is working as intended before integrating all the components together for final testing and implementation.

Another reason we chose this modular approach was for ease of reconfigurability, if in a future adaptation of this project, we need to change the functionality of one component, for example replacing the current cruise controller with a more complicated control system. Such changes to the design would be able to be applied with ease, and formally verified independently of the current system. This leads to the fact that the modular design approach adopted in this project is also much more simple for people to understand than alternate approaches containing more complex state machines.

Another interpretation we made for this project is that we assume that when the `Accel` and `Break` signals aren’t being emitted, they should take a value of completely zero. This may seem like a small change, but could end up critical for safety reasons. Although the design expects an emitted value for these signals each tick, the signal not being emitted could be evident of an underlying hardware error, such as the pedals being disconnected. If this is the case, our system could mitigate the consequences of these errors through this assumption.

We have also made another design decision for safety purposes, ignoring all of the inputs on the first tick. Often in hardware systems, there is a brief transient of garbage data upon startup, our choice to ignore the first tick of data can prevent any race conditions from arising if this does occur at minimal cost to the functionality of the rest of the program, because this program is assumed to run at a relatively high tick rate.

VII. CONCLUSION

This project presented a method of implementing a simplified cruise control system for a semi-autonomous vehicle using the reactive Esterel language predominately. It successfully regulates the car's speed when active and accurately activates and deactivates when necessary. The design utilised many of the important reactive features of Esterel; such as signals, aborts and concurrent execution. In addition to pure Esterel, it uses supplementary C function calls for complex mathematical data handling, and the project even features its own automated input vectoring. This design has yet to be formally verified in an actual vehicle, but has been successfully verified to follow all the relevant design constraints to be able to do so.

REFERENCES

- [1] K. Ogata, *Modern control engineering*. Prentice hall, 2010.